问题

问题1：运算时的数据强转

问题描述

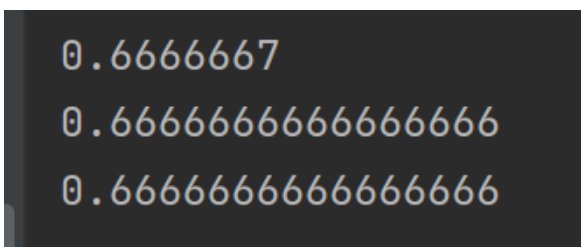
不同类型数据间的转换，中，是怎么进行转换的，如果一个int类型的变量和一个float类型的变量进行计算，结果是double还是float?

回答

使用如下代码：

```
1    public static void main(String[] args) {
2        int i = 1;
3        float f = 1.0F;
4        System.out.println((i+f)/3);
5        double d = 1.0;
6        System.out.println((i+d)/3);
7        System.out.println((i+f+0.0)/3);
8    }
```

输出为：



显然，**int类型与float类型运算的结果是float类型**，但是任何一个小数都会被认为是一个double类型，所以只要运算中混入了小数，那么结果就是double类型的

问题2：类的访问域

问题描述

这个对不对咧

主要是这道例题，从这张图片开始：

```
package p;
public class Demo{
        private int var1=1;
        int var2 = 2;
        protected int var3 = 3;
        public int var4 = 4;
             …
}
```

	var1	var2	var3	var4
package p1				√
package p		√	√	√
class newDemo1 extends Demo in p			√	√
class newDemo2 extends Demo in p1			√	√
class Demo	√	√	√	√

回答

类的访问域

	同一个类中	同一个包中	不同包的子类	不同包的无关类
public	✓	✓	✓	✓
protected	✓	✓	✓	
无（空着不写）	✓	✓		
private	✓			

这张图可解

值得一提的是：

1. protected使得子类，即使在不同包中，也能访问
2. default使得不同包中的子类不能访问，其实际上也就是不能让不同包的类进行访问

问题3：白盒测试的一些定义问题

问题描述

白盒测试中的测试漏洞不太了解，语句覆盖感觉有点问题，需要问老师

回答

白盒测试

首先，判定覆盖肯定是一种最强的**语句覆盖**，所有判定分支都走过了可不就是所有语句都执行了。然后条件覆盖要求覆盖各个条件表达式的所有可能结果。

判定覆盖和条件覆盖的区别是**粒度不同**，判定覆盖着眼于if表达式整体是true还是false，这两种至少执行一次，条件覆盖是if里面的简单判定条件都取到过true或者false，条件覆盖和语句覆盖乍一看没啥区别，实际上对于下面这个例子：

> 判断条件：
> `IF (A<10 OR B>10)`
> 当选择判断覆盖式，则整个判断式要有true和false两个结果的两组测试数据。
> 但是对条件覆盖来说，可以选择都为true的测试数据，如A<5,B<5（true）和A>15,B>15(true)，我们覆盖到了判断条件所有的可能性，但是两组数据的结果都为true，所以并没有覆盖到所有的分支，这就是条件覆盖与判断覆盖的区别。

条件组合覆盖只需要使得每个判定表达式里的简单判定条件的可能取值组合情况都执行一次，比如，一个if语句里面有两个简单判定条件，那么对于这个判定表达式就需要有TF、TT、FF、FT这四种条件（和其他的判定表达式的组合无所谓）

此外，测试漏洞指的是在测试过程中未能发现的程序缺陷或逻辑错误

此外！条件覆盖不能保证语句覆盖！所以PPT应该是有点问题的！

问题4：产品族和产品等级的定义

问题描述

产品族和产品等级的定义不太清楚

回答

产品族是指一系列相互关联、具有相似特点的产品。这些产品通常都由**同一家公司或组织**开发、生产和销售，并满足同一类顾客的需求。例如，苹果公司的产品族包括iPhone手机、iPad平板电脑、MacBook笔记本电脑、iPod音乐播放器等。

产品等级则是指同一产品族中具有不同功能和价格的产品。产品等级通常是根据产品的质量、性能、功能和价格等方面进行划分。例如，苹果公司的iPhone手机产品等级包括iPhone SE、iPhone 12、iPhone 12 Pro等

问题5：finally语句

问题描述

finally-总是会被执行，主要用于回收在try里打开的物力资源，只有finally执行完了之后才会执行try或者catch块中的return或者throw语句，如果finally中使用了return或者throw等终止方法的语句，则不会跳回执行，**直接停止，***这里不太清楚执行的具体流程*

回答

首先，如果try正常执行，finally里没有return语句，则会return try中return的值

如果try正常执行，finally里有return语句，则只会执行finally里的return

如果有try-catch-finally块，方法有返回值但只有try里写了return，则会报错

在上面的条件里如果catch里有return语句，则不会报错

如果显式地在try里的return语句之前抛出异常，则编译器会报错，return不可到达，去掉后，如果finally里没有return，则会执行catch块里的return，如果finally里有则会执行finally里的return

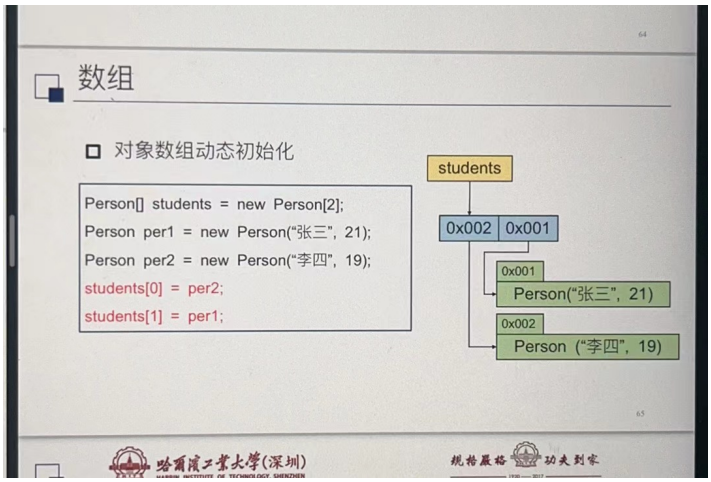
---

问题6：数组存储引用类型

问题描述

数组如何存储引用类型

回答



问题7：Runnable

问题描述

为什么Runnable更容易实现线程之间资源的共享

回答

因为Runnable接口对象是一个可共享的对象，通过将需要的资源封装在Runnable中，多个线程可以同时访问该对象，并通过加锁等机制保持数据的一致性

代码示例：

```
1 public class Counter implements Runnable {
2     private int count;
3     public synchronized void increment() {
4
```

```
        count++;
    }
    public synchronized int getCount() {
        return count;
    }
    public void run() {
        for (int i = 0; i < 1000; i++) {
            increment();
        }
    }
    public static void main(String[] args) {
        Counter counter = new Counter();
        Thread thread1 = new Thread(counter);
        Thread thread2 = new Thread(counter);

        thread1.start();
        thread2.start();
        try {
            thread1.join();
            thread2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Count: " + counter.getCount());
    }
}
```

可以看出，如果我们要共享资源，使用Runnable传入thread的方法确实还是比较方便的，这也意味着run方法要被执行两次，因此increment和getCount函数需要加锁（其实感觉increment不加也行），这样，只需要把counter传入线程，就实现了两个线程之间的资源共享。

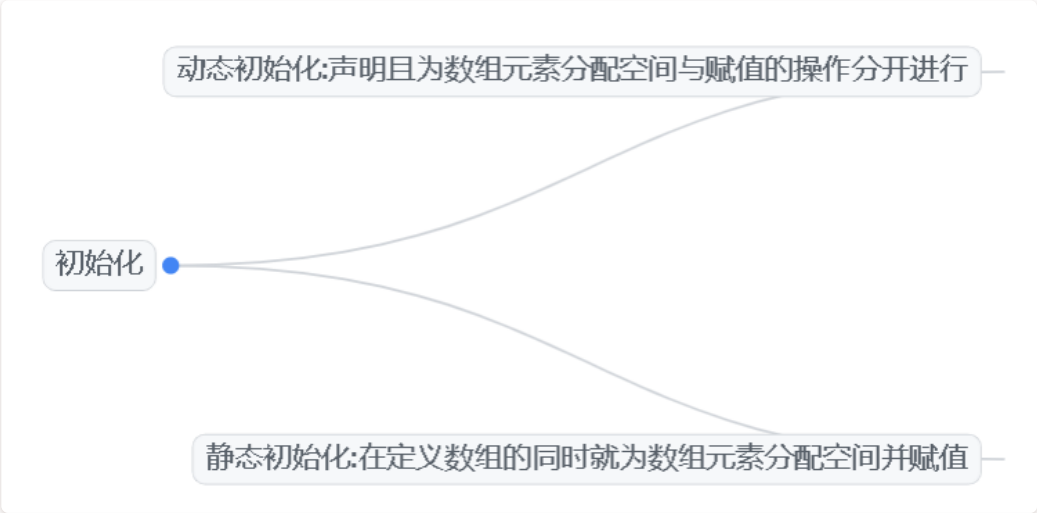
值得一提的是，这里的join的作用是

如果一个线程使用了 `.join()` 方法，那么**调用该方法的线程（即等待线程，这里指的是主函数）**会进入等待状态，直到被等待的线程执行完毕。

此外，如果一个对象有多个`synchronized`方法，它们之间不会共享锁，也就是说，**一个线程持有一个方法的锁并不会影响另一个线程对于另一个方法的访问**

问题8：数组初始化

问题描述



没有实际例子有点抽象

回答

静态初始化是指在定义数组时，同时为数组元素指定初值。例如：

```
1 int[] numbers = {1, 2, 3, 4, 5};
```

动态初始化是指在定义数组时，只指定数组的长度，而不为元素指定初值。例如：

```
1 int[] numbers = new int[5];
```

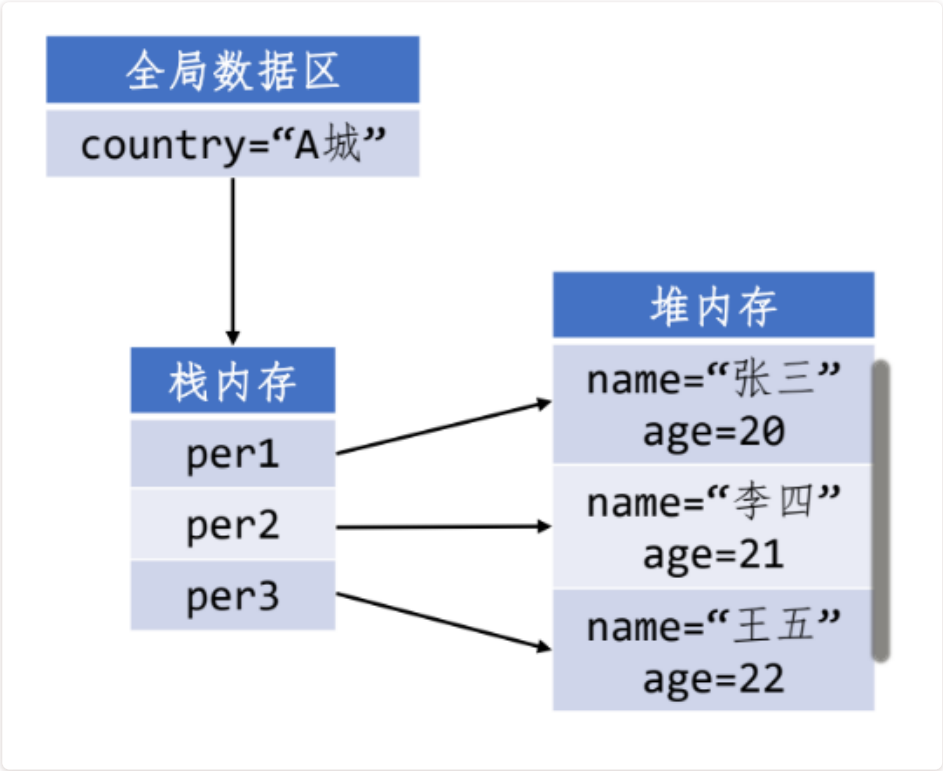
注意，**静态初始化只能在定义数组的时候使用，而动态初始化可以在任何时候使用，如果需要在程序运行的时候确定数组的长度，只能使用动态初始化**

```
int[] x = new int[2]{0,1,2};
```

这种动态 + 静态的初始化方式就不行了，有点逆天

问题9：全局数据区、栈、堆

问题描述



堆内存栈内存都是啥，全局数据区又是啥

回答

全局数据区用来存储全局变量和静态变量，它们的生命周期和整个程序的生命周期相同，程序启动时被分配，程序结束时被释放

堆内存存储java对象，程序启动时分配，程序结束时释放，优点是可以动态地分配和释放内存，但是需要GC，所以性能上会有一定的损失

栈内存用于存储**基本数据类型**和**对象的引用。**特点是空间大小固定，分配和释放内存的速度都很快

(？？？）问题10：handle

问题描述

网络上传输的都是字节流！

客户端创建Socket，与服务器连接，获取输入输出流，送入handle中进行传输

```java
public class Client{
    public static void main(String[] args) throws IOException {
        Socket sock = new Socket("localhost", 6666)
        //尝试连接指定服务器和指定端口
        try(InputStream input = sock.getInputStream()){
            try(OutputStream output = sock.getOutputStream()){
                handle(input, output);
            }
        }
        sock.close();
        System.out.println("disconnected.");
    }
}
```

回答

查了下，暂时还没发先有啥用

问题11：getDeclaredConstructors

问题描述

既然是获得所有定义的构造函数，那应该返回的是一个数组，为什么实际使用的时候返回的不是一个数组呢？

回答

在Singleton中，我定义了：

```java
public class Singleton {
    public int haha;
    private static Singleton singleton = new Singleton();
    private Singleton(){}
    private Singleton(int haha){this.haha = haha;}

    public static Singleton getInstance(){
        return singleton;
    }
}
```

这里我有两个构造方法，顺序放置，然后我再调用getDeclaredConstructors：

```java
import java.lang.reflect.Constructor;

public class BrokeSingleton {
    public static void main(String[] args) {
        Singleton singleton = Singleton.getInstance();
        Class cl = singleton.getClass();
        try {
            Constructor[] cons = cl.getDeclaredConstructors();
            cons[0].setAccessible(true);
            cons[1].setAccessible(true);
            Singleton newSingleton = (Singleton) cons[0].newInstance();
            Singleton newSingleton2 = (Singleton) cons[1].newInstance(123);
            System.out.println(singleton);
            System.out.println(newSingleton);
            System.out.println(newSingleton2);
        }
        catch(Exception e){
            e.printStackTrace();
        }
```

```
    }
}
```

这里可以看出，我使用了getDeclaredConstructors之后，需要定义一个Constructor的数组来存储，然后要对**这个数组里的每一个con对象都要设置访问来绕过安全检查机制，然后再调用对应的构造方法**，值得一提的是，这里在获取构造方法的时候应该是按照Singleton里定义的顺序来的

问题12：BufferedReader.read()

问题描述

BufferedReader的read方法的返回值是什么?

回答

```
char x = stdin.read();
```

首先不是char，是一个int类型的变量，如果要赋值给一个char类型的变量需要进行强制类型转换

实际上read()方法返回的是一个整数，该整数表示读入的字符的Unicode码点。它不一定是ASCII码，因为Unicode包含了ASCII，如果读取的字符是ASCII码可以表示的，那么该整数的值与ASCII码相同，**如果读取的字符不是ASCII码，则该整数的值将大于127**

问题13：读写文件怎么操作呢

问题描述

关于文件读写那章属于是一窍不通了，虽然序列化会一点但是单纯的文件读写属于是狗屁不通了

回答

```java
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

public class PathAndFile {
    public static void main(String[] args){
        Path absolute = Paths.get("D:\\JAVACODE\\LessonProblems\\src\\hello.txt");
        Path relative = Paths.get("hello.txt");
        try {
            byte[] bytes = Files.readAllBytes(absolute);
            for(byte a: bytes) {
                System.out.println(a);
            }
        }
        catch (Exception e){
            e.printStackTrace();
        }
    }
}

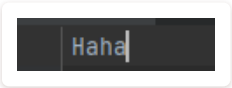
```

首先我们讨论一下Path的获取，绝对路径和相对路径，这里要使用Paths.get，然后路径分隔符我使用了一个\，这个问题在于\是转义字符的开头，**所以要用两个\来当路径分隔符，然后我使用Files类的readAllBytes方法，注意这种结尾带s的一般是返回一个数组**

接下来我们考虑写入：我们分别使用如下语句：

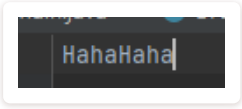
```java
String content = "Haha";
Files.writeString(absolute, content, StandardCharsets.ISO_8859_1);
System.out.println(content);
```

因为没有使用`StandardOpenOption.APPEND`参数，导致会覆盖原文件，改写后的文件长成这样：

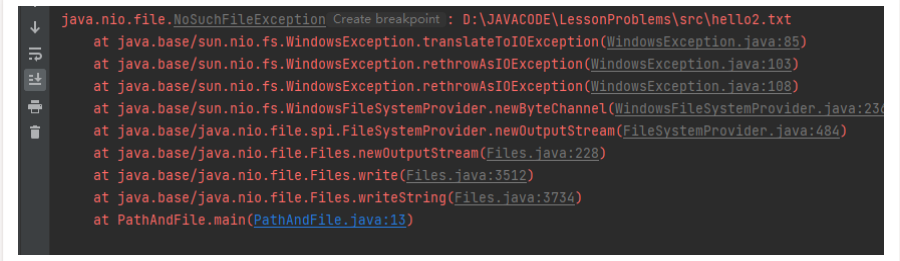


如果我们加上了这个参数，则会有

```
String content = "Haha";
Files.writeString(absolute, content, StandardCharsets.ISO_8859_1, StandardOpenOption.APPEND);
System.out.println(content);
```



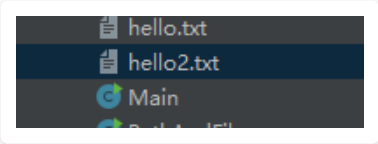
如果这个文件不存在，会发生什么呢?



答案是报错，但是如果我们提前捕获了这个异常，可以使用createFile方法创建这个文件

但我们还可以使用File.exists()来判断这个文件是否存在，见下文：

```
try {
    String content = "Haha";
    if(Files.exists(absolute)) {}
    else{
        Files.createFile(absolute);
    }
    Files.writeString(absolute, content, StandardCharsets.ISO_8859_1, StandardOpenOption.APPEND);
    System.out.println(content);
}
```



可以看到已经新建了一个文件并且已经把内容写入进去了

这部分就到此为止

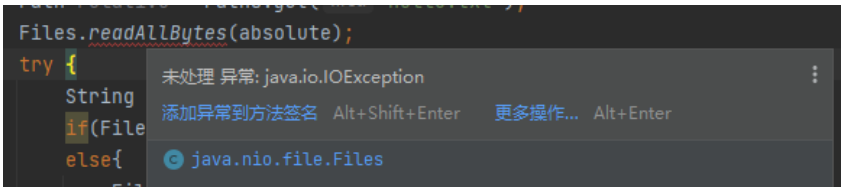
现在想想上面的if代码还是有点臃肿，这里做一点优化：

```
try {
    String content = "Haha";
    if(!Files.exists(absolute)) {
        Files.createFile(absolute);
    }
    Files.writeString(absolute, content, StandardCharsets.ISO_8859_1, StandardOpenOption.APPEND);
    System.out.println(content);
}
```

问题14：有些方法必须在try里实现

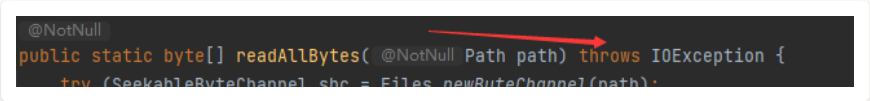
问题描述

有些方法必须写在try里，比如序列化中的readObject()，以及文件读写中的readAllBytes：见下图



回答

throw和throws的用法首先我们要知道这里有一个throws，之所以要写在try里，是因为这些语句可能抛出异常，比如我们检查readAllBytes的定义：



就可以看到其抛出了一个IOException异常

问题是，如果这个函数里会抛出异常，那要不要写在try里呢?

```
int nanbeng;
1 个用法  1 个相关问题
public MyException(int x) throws Exception
    if(x==1){
        nanbeng = x;
    }
    else{
        throw new Exception("haha");
    }
}
```

我们先定义一个MyException类以及一个抛出异常的构造方法

可以看出，这时候如果我们在主函数调用这个方法，会报错

```
MyException myException = new MyException(2);
```

这时候如果我们删掉了throws，这时候这个方法内部就报错了，如果我们在方法里面使用try

```
public MyException(int x){
    if(x==1){
        nanbeng = x;
    }
    else{
        try {
            throw new Exception("haha");
        }
        catch (Exception e){
            System.out.println(123);
        }
    }
}
```

```
MyException myException = new MyException(2);
```

这时候就会发现不报错了，因为try已经被写在MyException类里面了，但这个时候我们再加上

```
public MyException(int x) throws Exception{
```

这时候Main又会报错，所以总结：**Main调用一个签名里有throws的方法时必须要写在try块里，如果没有throws方法则不用，因为如果不在这个方法内部使用try来处理错误，那么编译还是通过不了**

问题15：继承与访问域

问题描述

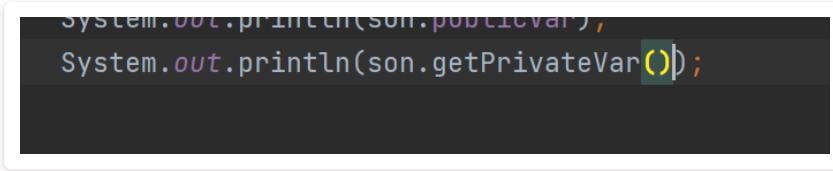
我在飞机大战中发现子类不能直接使用父类的属性和方法，只要父类的属性和方法不是protected或public，那这些变量会存在吗，如果没有调用父类的无参构造，那还可以通过super.属性来改变或访问其值吗

回答



首先，如果父类中某个变量或者方法是private的，那么子类不能直接通过.成员来获得这个变量





但是我写一个方法就可以了

这时候发现子类的代码：

```
public class Son extends Father{
    public int sonVar;
    Son(int x){
        this.sonVar = x;
    }
    public static void main(String[] args){
        Son son = new Son(400);
        System.out.println(son.sonVar);
        System.out.println(son.protectedVar);
        System.out.println(son.publicVar);
        System.out.println(son.getPrivateVar());
    }
}
```

是没有使用无参构造的子类的无参构造会自动调用父类的无参构造

也就是如果没使用子类的无参构造，那么也会调用父类的无参构造，我们再进一步探究，如果父类也没有无参构造呢?

```
D:\JAVACODE\TestQuestions\src\Son.java:3:15
java: 无法将类 Father中的构造器 Father应用到给定类型;
  需要: int
  找到:    没有参数
  原因: 实际参数列表和形式参数列表长度不同
```

就会出现编译错误，因为父类不再有无参构造了

```
Son(int x){
    super();
    this.sonVar = super.privateVar;
}
```

同时可以看到，super.变量的调用方式仍然不能调用父类的中的私有变量

问题16：Object.wait()

问题描述

Object.wait()是怎么使用的?

回答

```
1  Object lock = new Object();
2        Thread t1 = new Thread(() -> {
3            synchronized (lock) {
4                try {
5                    System.out.println("Thread 1 b
efore wait");
6                    lock.wait();
7                    System.out.println("Thread 1 a
fter wait");
8                } catch (InterruptedException e) {
9                    e.printStackTrace();
```

```
            }
        }
    });
```

首先，**wait必须要在同步块中使用**，**join可以在任意位置调用**，wait是一个对象级别的方法，必须在一个对象上调用，并且会释放该对象的锁，而join()是一个线程级别的用法，必须在一个线程对象上调用，表示当前线程等待该线程执行完毕。

wait()必须等待其他线程调用**相同对象上的notify()或notifyAll()方式来唤醒它，join()会等待目标线程执行完毕之后自动返回**

在Java中，当一个线程调用对象的wait()方法时，它会释放该对象的锁，并且会使该线程进入等待状态，直到被其他线程唤醒。在等待期间，该线程不再持有该对象的锁，因此其他线程可以在此期间获得该对象的锁并调用对象的其他方法。

假设在等待期间，另一个线程正在调用该对象的另一个synchronized方法，那么该线程仍然持有该对象的锁，因此其他线程无法同时访问该对象的其他synchronized方法。只有当该线程退出该方法并释放该对象的锁时，其他线程才能获取该对象的锁并调用该对象的其他synchronized方法。

需要注意的是，如果多个线程同时调用该对象的不同的synchronized方法，并且它们之间存在互相等待的情况，那么可能会出现死锁的情况。因此，在使用synchronized关键字时，需要仔细考虑各个线程之间的依赖关系，避免出现死锁等问题。

问题17：interrupt

问题描述

我们知道一个线程调用interrupt()方法时，其中断状态将被置位，那么具体怎么使用interrupt来停止一个线程呢？

回答

```java
public class MyThread extends Thread {
    private boolean running = true;

    @Override
    public void run() {
        while (running) {
            System.out.println("线程正在执行...");

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                System.out.println("线程被中断，即将退出...");
                running = false;
            }
        }
    }

    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();

        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        t.interrupt();
    }
}
```

可以看到，这里是在sleep的过程中等待interrupt，如果被interrupt了，则会抛出一个InterruptedException并等待处理，然后将标志变量置位，从而退出循环，但是还可以用下面的这种方法：

```
while (!Thread.currentThread().isInterrupted()) {
    System.out.println("线程正在执行...");

    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        System.out.println("线程被中断，即将退出...");
        Thread.currentThread().interrupt();
    }
```

这种就显得比较麻烦了（？？？）

问题18：对象模式类模式

问题描述

这两个模式有什么区别？不是非常清晰

回答

首先，在我们课程学过的设计模式中，

除了工厂方法、模板方法以外都是对象模式：处理对象之间的关系，而工厂方法和模板方法都是类模式：处理类和子类的关系，通过继承来建立，编译时刻便确定下来了。

至于**抽象工厂为什么是对象模式**，这是因为抽象工厂提供一系列接口用于创建相关或相互依赖的产品族，而不需要指定具体的类，这意味着它强调对象之间的交互。***(这个得靠一种感觉)***

而**工厂模式**强调如何更好组织和设计类的功能，比如我要一个比萨分店，那么我就需要一个具体的比萨分店类和具体的比萨类，把创建的流程写死等，重点是类的静态设计，而不是对象交互组合

模板模式就不用说了，纯纯的类模式

问题19：Graphics对象

问题描述

JAVA中，所有的绘制都必须通过Graphics对象来完成，所以具体是怎么完成的，没太理解，以及2D图形的绘制什么的

回答

```
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.RenderingHints;
import java.awt.geom.Ellipse2D;
import javax.swing.JFrame;
import javax.swing.JPanel;

public class DrawCircle extends JPanel {

  public void paint(Graphics g) {
    super.paint(g);
    Graphics2D g2d = (Graphics2D) g;

    g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
    //抗锯齿，不用太在意

    g2d.setPaint(Color.gray);
    g2d.fill(new Ellipse2D.Double(50, 50, 300, 300));

```

```
    g2d.setPaint(Color.yellow);
    g2d.fill(new Ellipse2D.Double(100, 100, 200, 200));

    g2d.setPaint(Color.gray);
    g2d.fill(new Ellipse2D.Double(150, 150, 100, 100));
  }

  public static void main(String[] args) {
    JFrame frame = new JFrame("DrawCircle");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.add(new DrawCircle());
    frame.setSize(400, 400);
    frame.setLocationRelativeTo(null);
    frame.setVisible(true);
  }
}
```

首先主函数里要先设置一个新的JFrame，然后要设置默认的关闭操作，接下来要setSize，即窗口大小，然后添加一个JPanel的组件，最后设置setVisible为true

在新建一个JPanel对象的时候，我们需要重写其paint方法：它的方法头长起来像这个：public void paint(Graphics g)。然后，我们可以选择使用super.paint(g)的方法，然后如果要进行2D绘图，则可以新建一个Graphics2D的对象，用来将Graphics对象g进行向Graphics2D类型的强转，接下来可以把这个g2d当做一个“画笔”，可以设置颜色（setPaint)，也可以去画一些东西（fill)

当然，如果不用Graphics2D，我们也可以使用

```
  public void paint(Graphics g) {
    super.paint(g);
    // 设置绘图颜色
```

```
        g.setColor(Color.blue);
        // 绘制矩形
        g.drawRect(50, 50, 200, 100);
        // 绘制圆形
        g.drawOval(100, 100, 100, 100);
        // 绘制直线
        g.drawLine(50, 150, 250, 150);
    }
```

这样的绘图方法，即直接调用API进行绘图，当然最开始还是要加上**super.paint(g);语句，其作用是清除绘图区域，这个方法实际上是JComponent类的paint方法**

`super.paint(g)`语句在`JPanel`的`paint`方法中的作用是清除绘图区域，以确保之前绘制的图形不会影响后续的绘制操作。这个语句是必须的，否则在界面重绘时，可能会出现之前绘制的图形残留在界面上的情况。

最后，对paint方法进行一个补充，初始化JComponent的子组件时会自动调用这个方法，多的在这里先不谈了

问题20：子类重写+重载

问题描述

如果在子类中定义一个方法，其名称、参数、返回类型正好与父类中某个方法相同，那么可以说是重写，那如果同时有重载，或者重载了父类的方法呢

回答

我们首先来看看Father类：

```
public class ReFather {
    public void haha(){
        System.out.println(100);

```

```
    }
}
```

再来看看Son：

```
public class ReSon extends ReFather{
    public void hahaSon(int x){
        System.out.println(200);
    }
    public static void main(String[] args)
{
        ReSon reson = new ReSon();
        reson.haha();
        reson.hahaSon(100);
    }
}
```

首先，这里如果要用多态会出问题在使用多态，即父类引用指向子类引用时，该引用只能访问父类和子类共有的方法和属性

如果确实需要访问子类特有的变量，可以通过将父类引用强制转换为子类引用来实现。例如：

```
Parent parent = new Child();
Child child = (Child) parent;
int childVariable = child.childVariable;
```

之后我们将不从多态出发

```
    public void haha(int x){
        System.out.println(200);
    }
    0 个用法
    public static void main(String[] args){
        ReSon reson = new ReSon();
        reson.haha();
        reson.haha( x: 100);
    }
}
```

可以看到，这里应该是已经继承了父类中的haha方法，并且在子类中又对这个方法进行了重载

但是如果我把父类改成抽象类，haha改成抽象方法，就会报错了



因为这个时候我们实现父类中指定参数或指定返回类型的抽象方法，因此这时不是一种重载，而是实打实的Error了

```
    0 个用法   MaxFan1337 *
    public static void main(String[] args){
        ReFather reson = new ReSon();
        reson.haha();
        reson.haha(100);
    }
}
```

此外，如果父类是抽象类，这种多态的方式仍然不能应用于子类独有的字段或方法

问题21：泛型通配符

问题描述

泛型通配符应该怎么用呢

回答

我们的泛型系统里，不能完全不加限制，比如我现在创建了一个List<T>类型的泛型，这是一个列表，无论什么类型都能加以存储，但是现在我要为这种类型写一个遍历函数，传参的时候要允许传入多种基类型的List<>类型（比方说，如果我传入List<String>，这是一个写死的类型），如果我们希望多样化，我们可以使用List<?>，但是这个太广泛了，再比如，LIst<Object>是一个针对Object的List，但是其不能接受所有的类型

看下面这个例子：

```
import java.lang.reflect.Array;
import java.util.ArrayList;

public class FanXing {
    public static void main(String[] args){
        ArrayList<Integer> intArray = new ArrayList<>();
        intArray.add(1);
        intArray.add(2);
        ArrayList<String> stringArray = new ArrayList<>();
        stringArray.add("haha");
        stringArray.add("wtf");
        printArray(intArray);
    }
    public static void printArray(ArrayList<?> array){
        for(Object num: array){
            System.out.println((int)num);
        }

```

```
    }
}
```

你会发现，唉？如果我传入的是String基类的ArrayList会怎么样呢



哦豁，出问题了，至少格式肯定是不匹配的

但是这就对应了：为什么要有通配符

严格的泛型类型系统一旦制定无法改变，因此我们要寻求改变（这里代表了使用？），但是我们也不能真的完全随意，还是要施加控制，接下来再写一个例子

```
        array1.add(new Class1( x: 1));
        array1.add(new Class1( x: 2));
        ArrayList<Class1_1> array2 = new ArrayList<>();
        array1.add(new Class1_1(1,2));
        array1.add(new Class1_1(2,3));
        ArrayList<Class2> array3 = new ArrayList<>();
        array1.add(new Class1( x: 3));
        array1.add(new Class1( x: 4));
        printArray(array3);
    }
    1 个用法  新 *
    public static void printArray(ArrayList<? super Class1_1> array){
        for(Object num: array){
            System.out.println(num);
        }
    }
```

这里Class1_1继承了Class1，我在printArray方法的参数中使用了? super Class1_1，这会导致编译器在检查printArray(array3)语句的时候会抛出错误，因为array3不在? super Class1_1的范围内

问题22：ArrayList的存储方式迭代器问题

问题描述

迭代器初始化的时候指向的是谁？使用了hasNext()或者next()之后呢，如果指向的数组已经改变了呢？比如ArrayList如果Remove了一个元素，这个时候迭代器又指向哪里呢？

回答

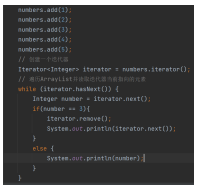
在Java中，`ArrayList`是一个动态数组，它可以根据需要动态地增加或删除元素。`ArrayList`底层实现是一个数组，这个数组是一个Object类型的数组，也就是说，`ArrayList`可以存储任意类型的对象，包括Java中的基本类型和自定义类型。

在`ArrayList`中，当元素数量超过数组容量时，会自动扩容数组，通常扩容的大小为原来数组大小的1.5倍左右，具体扩容大小可以通过`ensureCapacity`方法设置。当删除元素时，`ArrayList`会自动压缩数组，即将元素**向前移动**，填补空出的位置，以保证数组的连续性。

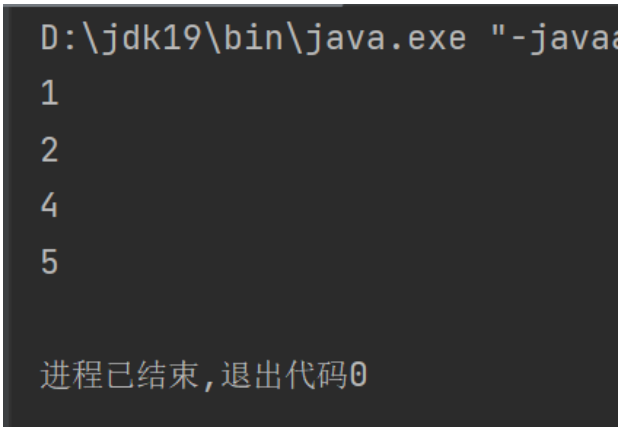


完美解决，我们再看看怎么用迭代器来删除ArrayList中不满足条件的元素

首先如果我们使得iterator=array.iterator()，这句话如果执行了第二遍，会重置iterator的指向位置，即可以做到反复遍历



```
numbers.add(1);
numbers.add(2);
numbers.add(3);
numbers.add(4);
numbers.add(5);
// 获取一个迭代器
Iterator<Integer> iterator = numbers.iterator();
// 用迭代器从前往后遍历集合并删除元素
while (iterator.hasNext()) {
    Integer number = iterator.next();
    if(number == 3){
        iterator.remove();
        System.out.println(iterator.next());
    }
    else {
        System.out.println(number);
    }
}
```



```
D:\jdk19\bin\java.exe "-javaa
1
2
4
5

进程已结束,退出代码0
```

我们来解构这个过程，首先，iterator指向array的第一个元素之前，while先判断其是否还跟着元素，如果是，则使用next，next获取iterator目前所在位置的下一个元素，并将iterator前移（第一步结束后，iterator在1和2的中间），然后打印这个next的值，当iterator在2和3之间的时候，此时number为3，iterator在3和4之间，然后number==3，iterator使用remove，即删除iterator后面的元素（3），此时让整个ArrayList后面的内容和iterator一起前移，这时候iterator指向的就是2和4之间了，下一步也就是要读取4了

问题23：静态泛型方法

问题描述

怎么用，注意点

回答

```
public static int fun(K k){
    System.out.println(k);
}
```

这里是不能使用的，因为静态方法无法从上下文引用非静态类型变量K

这个时候我们需要在static后面加一个<K>来代替这个静态方法参数列表里的K，即

```
public static <K> int fun(K k){
    System.out.println(k);
    return 1;
}
```

这样就不会报错了

```
GenericStatic<String> instance = new GenericStatic<>( k: "haha");
```

此外，这里之后的尖括号不用进行指定，可以理解为前一个<>里的String已经指定了K的类型

```
public class GeneticStaticMain {
    public static void main(String[] args){
        GenericStatic<String> instance = new GenericStatic<>( k: "haha");
        GenericStatic<Integer> anotherInstance = instance.<Integer>fun( k: 100);
        System.out.println(instance.k);
        System.out.println(anotherInstance.k);
    }
}
```

这真是有够抽象的

结果是

```
haha
12
```

不用太过在意这种写法，毕竟确实是有点过于抽象了，但是值得一提的是中间有个重复

操已经被绕进去了，下次再解决吧（？？？）

（？？？）问题24：泛型通配符++

问题描述

来自试卷上

回答

问题25：内部类问题

问题描述

- 我们创建Father类，在该类中添加strong()方法；同样的，创建Mother类，在该类中添加smart()方法，代码如下：

```
public class Father{
    public int strong(){
        // 强壮指数
        return 9;
    }
}
```

```
public class Mother{
    public int smart(){
        // 聪慧指数
        return 8;
    }
}
```

```
1. public class Son {
2.     // 内部类继承Father类
3.     class Father_Inner extends Father {
4.         public int strong() {
5.             return super.strong() + 1;
6.         }
7.     }
8.     // 内部类继承Mother类
9.     class Mother_Inner extends Mother {
10.        public int smart() {
11.            return super.smart() + 2;
12.        }
13.    }
14.     public int getStrong() {
15.         return new Father_Inner().strong();
16.     }
17.     public int getSmart() {
18.         return new Mother_Inner().smart();
19.     }
20. }
```

34

起因是这里，可以解释一下吗

回答

可以理解为，我想让儿子继承父亲和母亲的特征，但是不能多继承，所以创建了两个Father_Inner和Mother_Inner，其本质是两个Son的副本，为了表示所以使用了内部类的命名格式，其继承了父类的操作，可以当做两个独立的类对待，要重写重写，要干啥干啥，但是如果我要实现子类独特的操作的时候，这个操作如果是来自父类的行为，那么我需要创建一个实例来操作——这里的return new Father_Inner()就是一个典型的先创建实例再调用的行为，实际上我们还可以先在子类中创建好实例（视代码的复杂程度决定）